RISCVMamba

Individual Task Allocation and Checklist

Sahasra — HARDWARE (RTL AND FPGA)

Main Responsibility: Design the RISCVMamba hardware in Verilog, integrate it with the RISC-V processor, and implement it on the FPGA.

1. Setup and Onboarding

- Install Vivado and CFU Playground.
- Build the template project and run it in Renode or Verilator simulation.
- Read the CPU–CFU interface documentation.
- Write a half-page summary of the handshake signals.

2. Hardware Preparation

- Build a basic "Hello CFU" unit that performs one instruction, such as adding two registers.
- Synthesize the stock SoC for the XC7A100T FPGA in Vivado.
- Record LUTs, DSPs, BRAM and achieved clock frequency.
- Estimate DSP and BRAM requirements for RISCVMamba.
- Send the board request for Arty A7-100T, with Nexys Video as the backup option.

3. ISA Design Support

- Review each custom instruction for hardware feasibility.
- Estimate latency, DSP usage, BRAM usage and critical-path impact.
- Write the microarchitecture note, including the block diagram, `mb.scan` state machine, BRAM layout and pipeline stages.
- Start RTL implementation of `mb.mac4` and `mb.racc`.

4. RTL Design and Simulation

- Implement `mb.mac4` and `mb.racc` RTL with unit tests.
- Implement the nonlinear unit for `mb.exp`, `mb.silu` and `mb.softplus`.
- Implement the scan engine RTL with unit tests.
- Integrate RISCVMamba into the CFU Playground SoC.
- Simulate one complete token and verify its output against Spike.
- Run Vivado synthesis regularly.

- Record LUTs, flip-flops, DSPs, BRAM and worst timing slack.

5. FPGA Implementation

- Load and test the stock CFU Playground bitstream.
- Verify UART output and DDR memory.
- Build the RISCVMamba bitstream and achieve timing closure.
- Debug hardware failures using the Integrated Logic Analyzer (ILA), if required.
- Generate final resource, timing and power reports for both builds.

6. Final Evaluation

- Produce final hardware measurements and reports.
- Report FPGA resource usage, maximum clock frequency and estimated power.
- Optional: Implement four parallel scan lanes and deeper pipelining.

7. Paper-Related Tasks

- Help implement MAC-only and MAC-plus-nonlinear-unit comparison builds.
- Write the hardware design and evaluation sections of the research paper with Member B.
- Optional: Explore `mb.combine` and sequential-versus-parallel scan comparison.

Ranjith — ISA AND SOFTWARE

Main Responsibility: Define the RISCVMamba instruction set, implement it in Spike, develop the C inference engine and measure software performance.

1. Setup and Onboarding

- Build the RISC-V GNU toolchain for RV32IM with newlib.
- Build Spike and the proxy kernel.
- Run Hello World on Spike.
- Write a bare-metal program that reads the cycle and instruction-retired counters.
- Create the Git repository with the required project structure.

2. Golden Reference and Profiling

- Write the bare-metal C inference engine without `malloc`.
- Link model weights into the program as a binary blob generated from `.npy` files.
- Implement fixed-point arithmetic and lookup tables for exp, SiLU and softplus.
- Instrument each kernel using `instret` counters.
- Produce a profile table showing instructions per token, per kernel and percentage of total.

- Give Member C a skeleton for the NumPy step function, including signatures and one worked layer.
- Draft ISA v0 based on profiling and send it to Member A for review.

3. ISA Specification and Spike

- Write ISA specification v1, including instruction encodings, operand packing, rounding, saturation, internal state and latency targets.
- Implement `riscvmamba_ref.c`.
- Implement the RISCVMamba extension in Spike, including readable disassembly names.
- Write `riscvmamba.h` intrinsics using inline assembly and `.insn`.
- Port all relevant kernels to use the custom instructions.
- Check output against the fixed-point C baseline and NumPy reference.
- Report instruction counts for the baseline, individual instruction groups and full RISCVMamba.
- Freeze ISA v1 in agreement with Member A.

4. Software Integration

- Port the C inference engine to the CFU Playground SoC.
- Prepare startup code, UART output, timers and weights in DDR.
- Build a measurement harness for cycles per kernel and per token.
- Write the demo application to accept input through UART and print generated tokens and cycle counts.
- Debug any mismatch between Spike and the simulated SoC with Member A.

5. FPGA Testing

- Run the software baseline and RISCVMamba builds on the FPGA.
- Confirm outputs match simulation bit-for-bit.
- Optimize software by packing four int8 weights per word and unrolling inner loops.
- Keep frequently used data in cache where applicable.
- Collect cycles per kernel and per token on hardware.

6. Final Evaluation

- Complete final performance measurements.
- Tag the final reproducible release as `v1.0`.
- Optional: Add an `rsqrt` instruction for RMSNorm, implement weight prefetching or prepare a student-conference paper.

7. Paper-Related Tasks

- Work with Member A on MAC-only and FPU comparison builds.

- Write the design and evaluation sections of the research paper with Member A.
- Optional: Explore `mb.combine` and sequential-versus-parallel scan comparison.

Neethu — MODEL AND REFERENCE

Main Responsibility: Manage the trained Mamba model, develop the trusted Python/NumPy reference, generate test vectors and verify accuracy.

1. Model Setup

- Locate the trained checkpoint and training configuration.
- Prepare a model card containing `d_model`, `n_layers`, `d_state`, `d_conv`, `expand`, `dt_rank`, input or vocabulary size, parameter count, task and accuracy metric.
- Export every weight tensor to its own `.npy` file.
- Create a list of exported tensor names and shapes.
- Load the weights into `mamba-minimal` or the original training code.
- Confirm that the exported model produces matching outputs.

2. Golden Reference and Quantization

- Complete the NumPy single-token step function using Member B's skeleton.
- Compare NumPy outputs against PyTorch on 20 sequences.
- Keep maximum absolute error below `1e-4`.
- Export test vectors for every stage of every layer over 10 tokens.
- Save test vectors as binary files with a README describing their layout.
- Write an accuracy script that scores implementations on 100 test samples.

3. ISA and Reference Support

- Write `gen_luts.py` to generate exp, SiLU and softplus lookup tables.
- Report maximum lookup-table error against NumPy.
- Run the accuracy script on Spike RISCVMamba outputs.
- Record accuracy compared with the floating-point model.
- Freeze model weights; do not retrain without team agreement.

4. RTL and SoC Verification

- Prepare 10 demo inputs or prompts and their expected reference outputs.
- Keep test vectors and the accuracy script synchronized with any agreed ISA changes.

5. FPGA Testing

- Run the accuracy script on 100 samples collected from the board's UART log.

- Evaluate both the software baseline and RISCVMamba build.

6. Final Evaluation

- Prepare the final accuracy comparison table for floating-point, Spike and FPGA outputs.
- Optional: Measure the same model on a laptop CPU and Raspberry Pi for comparison.

7. Paper-Related Tasks

- Choose and retrain the model for the agreed evaluation task, with Member B reviewing.
- Support the final accuracy evaluation.
- Optional: Benchmark the model on a laptop and Raspberry Pi.

Lakshmi — TESTING AND BENCHMARKS

Main Responsibility: Verify correctness, automate testing, track bugs and produce performance results and charts.

1. Setup and Onboarding

- Reproduce the toolchain setup on your own machine using Member B's written instructions.
- Identify and correct unclear setup steps.
- Set up a task board, such as GitHub Projects, containing all project tasks.

2. Golden Reference and Testing

- Write `compare.py` to read two output dumps.
- Calculate maximum error and mean error.
- Print PASS or FAIL based on the specified tolerance.
- Create the results sheet with columns for build, test, metric, value and commit.
- Run Member B's C inference engine on Spike against the test vectors.
- Record the results.

3. Instruction Testing

- Write directed tests for every custom instruction.
- Include typical, zero, maximum, minimum and saturation cases.
- Generate expected results using `riscvmamba_ref.c`.
- Create `run_all.sh` to build the project and run all Spike tests.
- Make the script print one PASS/FAIL line per test.

4. RTL and Regression Testing

- Extend `run_all.sh` to run RTL unit tests in Verilator.
- Add whole-SoC tests.
- Record every failure in the bug log.
- Run the complete regression suite regularly.
- Post PASS/FAIL summaries in the team channel.

5. FPGA Testing

- Support the board bring-up process.
- Write a step-by-step guide explaining how to run the demo.
- Run the benchmark matrix on the FPGA for the software baseline, each instruction group individually and full RISCVMamba.
- Perform three runs for each configuration.
- Fill in the results sheet.
- Generate performance charts.

6. Final Evaluation

- Rebuild `v1.0` from a clean Git clone.
- Run the full regression suite again to confirm reproducibility.
- Finalize the results sheet and charts.
- Ensure the report's performance numbers match the recorded results.

7. Paper-Related Tasks

- Produce all paper figures from the results sheet.
- Provide resource, timing, power and performance comparisons required by the paper.

Keerthi — DOCUMENTATION AND COORDINATION

Main Responsibility: Maintain project records, coordinate the team, prepare the report, presentations, figures and final demo materials.

1. Setup and Documentation

- Write a one-page summary of the Mamba paper, sections 1–3.
- Write a one-page summary of MARCA.
- Write a one-page summary of LightMamba.
- Write a one-page summary of the CFU Playground paper.
- Set up the Overleaf/LaTeX report template.
- Add chapter headings and a shared bibliography.
- Create the weekly notes page.
- Record project decisions in the milestone plan.

2. Background and Figures

- Draft the background chapter covering Mamba and RISC-V custom extensions.
- Draw the Mamba block dataflow figure for the report.

3. ISA Documentation

- Typeset the ISA specification as a report chapter.
- Prepare instruction encoding tables.
- Prepare operand-packing diagrams.
- Prepare project-review slides covering the problem, approach, ISA and Spike results.

4. Methodology and Hardware Figures

- Write the methodology chapter describing ISA design and Spike results.
- Redraw the microarchitecture figures using Member A's hardware sketches.

5. Implementation and Results

- Photograph the FPGA setup.
- Record a short demo video.
- Draft the implementation chapter.
- Draft the results chapter.

6. Final Submission

- Coordinate final measurements and reports with the other members.
- Prepare the complete report draft.
- Ensure every member reviews their own chapter.
- Prepare the poster.
- Prepare the final presentation slides.
- Prepare a viva question bank.
- Coordinate final submission and demo documentation.

7. Paper-Related Tasks

- Set up the IEEE two-column paper template.
- Complete the related-work section.
- Coordinate the paper draft with Members A and B.
- Edit the full paper draft.
- Arrange advisor review.
- Prepare the arXiv preprint when permitted by the selected venue's policy.

COMMON TEAM RESPONSIBILITIES

- Choose the evaluation task together: keyword spotting, ECG arrhythmia or human activity recognition.
- Maintain one shared Git repository.
- Attend regular team meetings and record decisions.
- Coordinate hardware and software integration.
- Ensure model outputs are checked against the trusted reference.
- Run automated tests and record all results.
- Keep model weights and the ISA specification consistent across implementations.
- Ensure every task is merged into the main repository and required tests pass.
- Keep the results sheet updated with measurements and the corresponding commit.
- Prepare the final demo, report, presentation, poster and viva materials.

FINAL PROJECT DELIVERABLES

- Trained Mamba model and exported weights.
- Python/NumPy reference implementation and test vectors.
- Fixed-point C inference engine.
- RISCVMamba ISA specification.
- Spike extension and instruction tests.
- Verilog RTL for the MAC, nonlinear and scan units.
- Verilator unit tests and whole-SoC simulation.
- FPGA bitstreams for the baseline and RISCVMamba builds.
- Automated regression and comparison scripts.
- Benchmark results, FPGA resource reports and accuracy evaluation.
- Final project report, presentation, poster, demo video and viva question bank.